torch.func.linearize#

https://pytorch.org/docs/stable/generated/torch.func.linearize.html

Description:

Returns the value of func at primals and linear approximation at primals. func (Callable) – A Python function that takes one or more arguments. primals (Tensors) – Positional arguments to func that must all be Tensors. These are the values at which the function is linearly approximated. Returns a (output, jvp_fn) tuple containing the output of func applied to primals and a function that computes the jvp of func evaluated at primals.

Function Signature

```
torch.func.linearize(func, *primals)[source]#
Parameters
Returns
Return type
```

Returns:

tuple[Any, Callable]

Code Examples

```
>>> import torch
>>> from torch.func import linearize
>>> def fn(x):
...     return x.sin()
...
>>> output, jvp_fn = linearize(fn, torch.zeros(3, 3))
>>> jvp_fn(torch.ones(3, 3))
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
>>>
```

Notes & Warnings

NOTE Note linearize evaluates func twice. Please file an issue for an implementation with a single evaluation.

Detailed Documentation

Documentation

Returns the value of func at primals and linear approximation at primals.

func (Callable) – A Python function that takes one or more arguments.

primals (Tensors) – Positional arguments to func that must all be Tensors. These are the values at which the function is linearly approximated.

Returns a (output, jvp_fn) tuple containing the output of func applied to primals and a function that computes the jvp of func evaluated at primals.

linearize is useful if jvp is to be computed multiple times at primals. However, to achieve this, linearize saves intermediate computation and has higher memory requirements than directly applying jvp. So, if all the tangents are known, it maybe more efficient to compute vmap(jvp) instead of using linearize.

linearize evaluates func twice. Please file an issue for an implementation with a single evaluation.